# Práctica N°4: Uso de Adafruit en ESP32 con comunicación Wi-Fi

## Índice


**1. Objetivos** **2**
- 1.1. Objetivo General . . . 2
- 1.2. Objetivos Específicos . . . 2

**2. Configuración de Adafruit IO desde la Web** **2**
- 2.1. Creación de cuenta y acceso al panel principal . . . 2
- 2.2. Obtención de la clave de API (Adafruit IO Key) . . . 3
- 2.3. Creación de los Feeds . . . 3
- 2.4. Diseño del Dashboard . . . 4

**3. Explicación del Código** **8**
- 3.1. Código completo . . . 8
- 3.2. Importación de librerías y configuración inicial . . . 11
- 3.3. Declaración de credenciales y configuración del sistema . . . 11
- 3.4. Configuración de pines y del ADC . . . 11
- 3.5. Conexión Wi-Fi . . . 12
- 3.6. Configuración del cliente MQTT . . . 12
- 3.7. Recepción de mensajes (Callback) . . . 13
- 3.8. Lectura del potenciómetro y publicación de datos . . . 13
- 3.9. Bucle principal y verificación . . . 13

**4. Resultados y Pruebas de Funcionamiento** **14**
- 4.1. Publicación de datos en Adafruit IO . . . 14
- 4.2. Control remoto del LED desde el Dashboard . . . 15

# 1. Objetivos

## 1.1. Objetivo General

Configurar y programar una placa ESP32 para establecer comunicación con la plataforma Adafruit IO mediante el protocolo MQTT, permitiendo el envío y recepción de datos entre un sensor local (potenciómetro) y un tablero virtual en la nube.

## 1.2. Objetivos Específicos

- Registrar y configurar una cuenta en Adafruit IO, creando los *feeds* necesarios para la comunicación con el ESP32.
- Implementar la conexión Wi-Fi del ESP32 y establecer un cliente MQTT utilizando las librerías de MicroPython.
- Programar el envío periódico de datos analógicos (lectura de potenciómetro) hacia la nube y la recepción de comandos para el control de un LED remoto.
- Analizar el flujo de comunicación entre el dispositivo y el servidor Adafruit IO, comprendiendo los procesos de publicación, suscripción y callback.

# 2. Configuración de Adafruit IO desde la Web

Antes de poder ejecutar el código en el ESP32, es necesario realizar la configuración de la cuenta y los elementos correspondientes en la plataforma **Adafruit IO**. Esta etapa permitirá establecer la comunicación entre el dispositivo físico y la nube mediante el protocolo MQTT.

## 2.1. Creación de cuenta y acceso al panel principal

Para iniciar, se debe acceder al sitio oficial de Adafruit IO (`https://io.Adafruit.com`) y crear una cuenta gratuita. Una vez registrado, el usuario puede ingresar al panel principal, donde se gestionan los *feeds*, *dashboards* y *keys* de acceso.

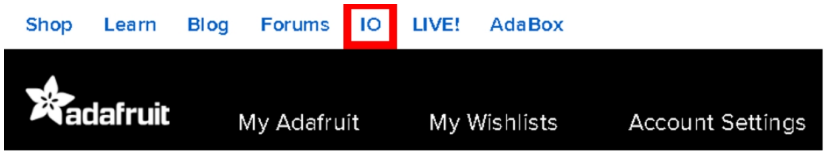



Figura 1: Pantalla inicial de Adafruit IO y registro de usuario.

## 2.2. Obtención de la clave de API (Adafruit IO Key)

Desde el menú lateral, se debe acceder a la opción **"My Key"**, donde se muestran las credenciales personales necesarias para la conexión MQTT. Esta clave se utilizará dentro del código en la variable `AdafRUIT_IO_KEY`, permitiendo autenticar la sesión del ESP32 con el servidor.

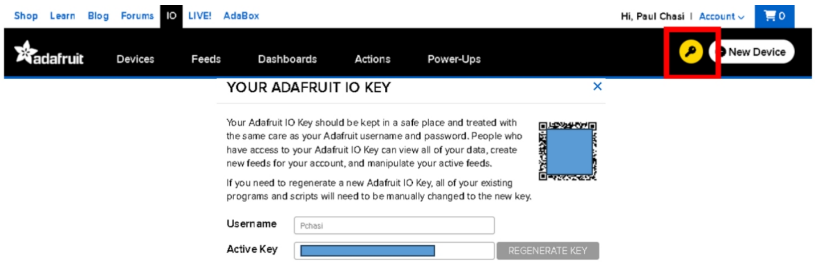



Figura 2: Creación de un nuevo feed en Adafruit IO.

## 2.3. Creación de los Feeds

Los *feeds* son los canales de comunicación entre la nube y el dispositivo. Para este proyecto se requieren dos:

- **esp32.pulsante**: para recibir valores (0 o 1) y controlar el encendido del LED.
- **esp32.potenciometro**: para enviar el valor leído del potenciómetro desde el ESP32 hacia la nube.

Estos se crean desde el menú **Feeds >Actions >Create a New Feed**, asignando los nombres exactos que luego se referencian en el código.

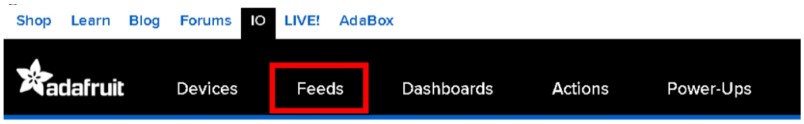



Figura 3: Ubicación de la clave de acceso (Adafruit IO Key) en la configuración de la cuenta.

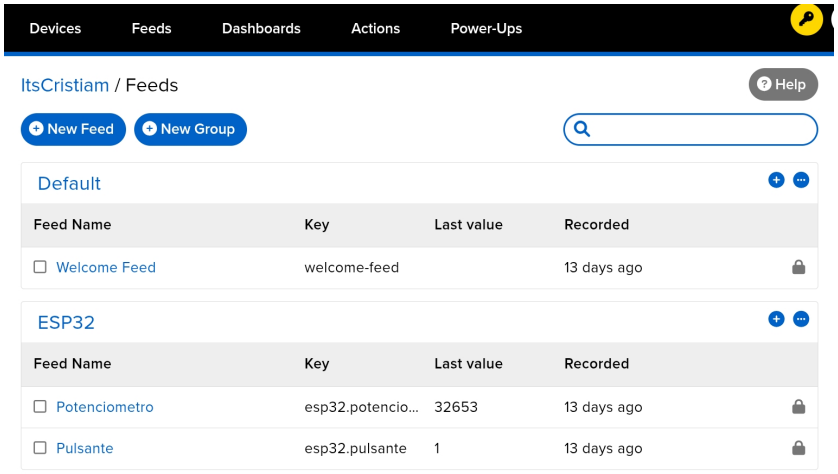



Figura 4: Ubicación de la clave de acceso (Adafruit IO Key) en la configuración de la cuenta.

## 2.4. Diseño del Dashboard

Una vez creados los feeds, se recomienda construir un *dashboard* para visualizar y controlar los datos. Desde **Dashboards** >**New Dashboard** se crea una interfaz donde se pueden agregar bloques interactivos:

- Un **slider o gauge** vinculado al feed `esp32.potenciometro` para observar el valor analógico enviado por el potenciómetro.
- Un **toggle button** asociado al feed `esp32.pulsante` para enviar el valor 0/1 al ESP32 y controlar el LED.

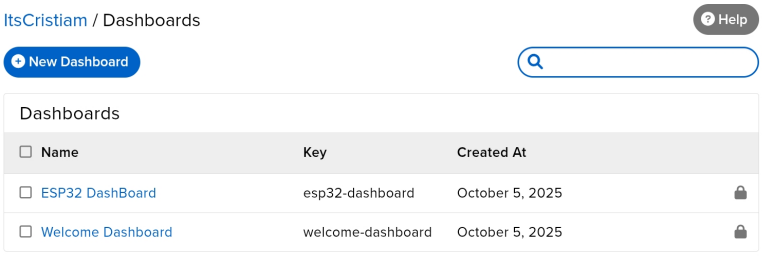



Figura 5: Configuración de bloques en el Dashboard y asociación con los feeds.




Figura 6: Configuración de bloques en el Dashboard y asociación con los feeds.

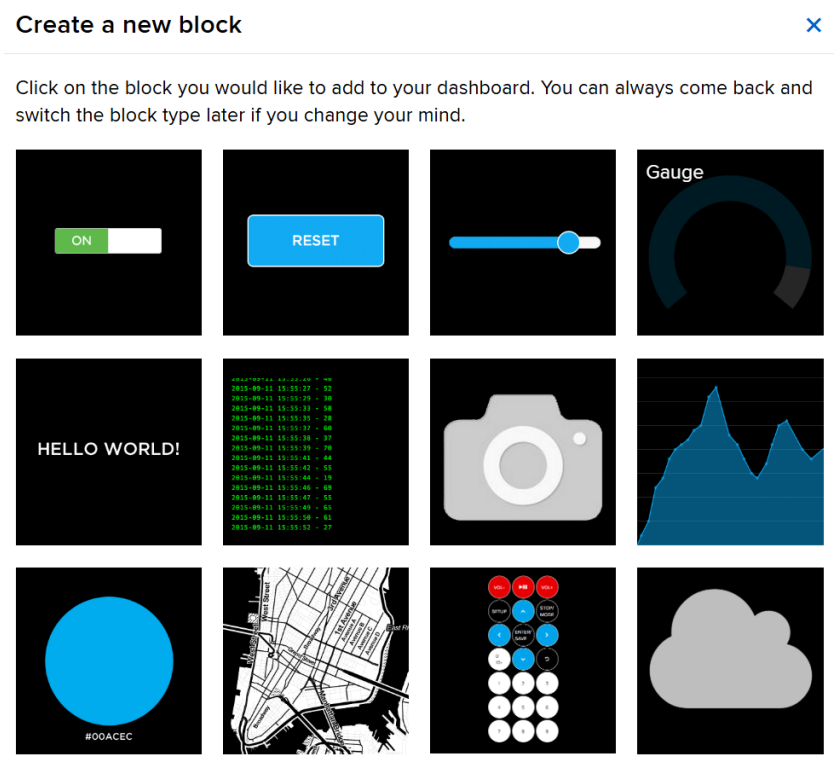



Figura 7: Configuración de bloques en el Dashboard y asociación con los feeds.

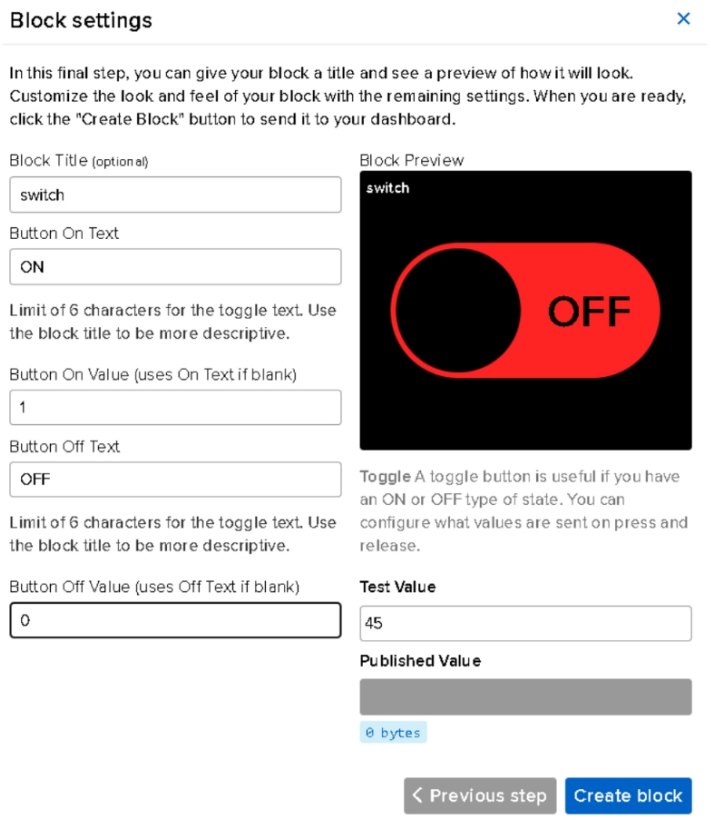



Figura 8: Configuración de bloques en el Dashboard y asociación con los feeds.

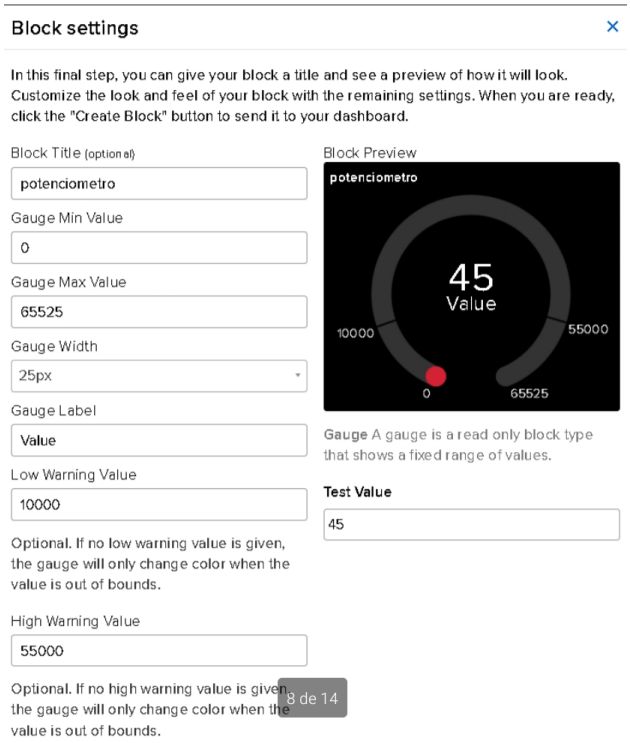



Figura 9: Configuración de bloques en el Dashboard y asociación con los feeds.

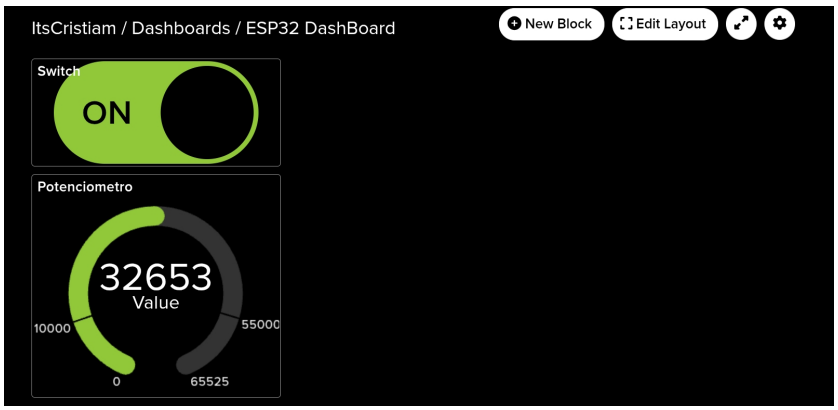



Figura 10: Configuración de bloques en el Dashboard y asociación con los feeds.

Con estos pasos completados, la plataforma Adafruit IO queda preparada para interactuar con el programa implementado en el ESP32, permitiendo

analizar posteriormente el funcionamiento del código, la lógica de conexión y el intercambio de datos mediante MQTT.

# 3. Explicación del Código

En esta sección se presenta el desarrollo del programa implementado en el microcontrolador **ESP32** para la comunicación con la plataforma **Adafruit IO**. El código fue escrito en **MicroPython** y utiliza el protocolo **MQTT** para enviar y recibir datos entre el dispositivo y la nube.

## 3.1. Código completo

```
# main.py - ESP32 MicroPython (Adafruit IO publish + subscribe)
# Requiere: umqtt.simple o umqtt.robust (subir a la placa o
    instalar via upip)

import network
import time
import sys
from machine import Pin, ADC

# --------- CONFIGURA AQUI ----------
SSID = "TU_RED_WIFI"
PASSWORD = "TU_CONTRASENA"

AdafRUIT_USERNAME = "TU_USUARIO_AdafRUIT"
AdafRUIT_IO_KEY = "TU_CLAVE_IO"
AdafRUIT_IO_SERVER = "io.adafruit.com"
AdafRUIT_IO_PORT = 1883

FEED_LED = "esp32.pulsante"
FEED_POT = "esp32.potenciometro"
# ----------------------------------

# Pines ESP32: LED y lectura analogica
LED_PIN = 2
POT_PIN = 34

led = Pin(LED_PIN, Pin.OUT)
adc = ADC(Pin(POT_PIN))
adc.atten(ADC.ATTN_11DB)
adc.width(ADC.WIDTH_12BIT)
```

```
30
31 # Intentar importar MQTT robusto
32 try:
33     from umqtt.robust import MQTTClient
34 except Exception:
35     from umqtt.simple import MQTTClient
36
37 # Funcion para leer ADC (0 a 65535)
38 def read_adc_u16(adc_obj):
39     try:
40         return int(adc_obj.read_u16())
41     except Exception:
42         v = adc_obj.read()
43         return int((v * 65535) // 4095)
44
45 # Generar topics MQTT
46 def topic_for(feedname):
47     return "{}/feeds/{}".format(AdafRUIT_USERNAME, feedname)
48
49 # Callback de mensajes recibidos
50 def mqtt_callback(topic, msg):
51     t = topic.decode()
52     m = msg.decode()
53     print("MSG IN ->", t, ":", m)
54     if FEED_LED in t:
55         if m.strip() == '0':
56             led.value(0)
57         else:
58             led.value(1)
59
60 # Conexion WiFi
61 def wifi_connect():
62     wlan = network.WLAN(network.STA_IF)
63     wlan.active(True)
64     if not wlan.isconnected():
65         print("Conectando a WiFi...")
66         wlan.connect(SSID, PASSWORD)
67         while not wlan.isconnected():
68             time.sleep(0.5)
69     print("Conectado a:", wlan.ifconfig()[0])
70
71 # Conexion MQTT y suscripcion
72 def mqtt_connect_and_subscribe():
```

```
    client_id = "esp32-" + str(int(time.time()))
    client = MQTTClient(client_id, AdafRUIT_IO_SERVER,
                        AdafRUIT_IO_PORT, AdafRUIT_USERNAME,
                        AdafRUIT_IO_KEY)
    client.set_callback(mqtt_callback)
    print("Conectando a Adafruit IO MQTT...")
    client.connect()
    client.subscribe(topic_for(FEED_LED))
    print("Suscrito a feed:", FEED_LED)
    return client

# Bucle principal
def main():
    wifi_connect()
    client = mqtt_connect_and_subscribe()
    pot_topic = topic_for(FEED_POT)
    last_publish = 0
    publish_interval = 2

    while True:
        client.check_msg()
        now = time.time()
        if now - last_publish >= publish_interval:
            v = read_adc_u16(adc)
            client.publish(pot_topic, str(v))
            print("Publicado:", v)
            last_publish = now
        time.sleep(0.1)

if __name__ == "__main__":
    main()
```

Listing 1: Código completo para conexión ESP32 – Adafruit IO en MicroPython.

## 3.2. Importación de librerías y configuración inicial

```
import network
import time
import sys
from machine import Pin, ADC
```

Estas librerías proporcionan las funciones básicas para el control del hardware y la comunicación:

- `network`: maneja la conexión Wi-Fi del ESP32.
- `machine`: permite acceder a los pines digitales y al ADC para leer el potenciómetro.
- `time` y `sys`: se emplean para temporización y manejo de errores.

## 3.3. Declaración de credenciales y configuración del sistema

```
SSID = "TU_RED_WIFI"
PASSWORD = "TU_CONTRASENA"
AdafRUIT_USERNAME = "TU_USUARIO_AdafRUIT"
AdafRUIT_IO_KEY = "TU_CLAVE_IO"
```

El usuario debe reemplazar estas variables por los datos de su propia red Wi-Fi y cuenta de Adafruit IO. Esto permite que el dispositivo se autentique correctamente al momento de publicar o suscribirse a los *feeds* definidos.

## 3.4. Configuración de pines y del ADC

```
LED_PIN = 2
POT_PIN = 34
led = Pin(LED_PIN, Pin.OUT)
adc = ADC(Pin(POT_PIN))
adc.atten(ADC.ATTN_11DB)
adc.width(ADC.WIDTH_12BIT)
```

Aquí se establecen los pines físicos del ESP32:

- El pin `GPIO2` controla un LED, que encenderá o apagará según la señal recibida.
- El pin `GPIO34` lee un valor analógico desde un potenciómetro.

La configuración del ADC define una resolución de 12 bits (0–4095) y una atenuación de 11 dB, lo que permite leer un rango aproximado de 0 a 3.6 V.

## 3.5. Conexión Wi-Fi

```
def wifi_connect():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        print("Conectando a WiFi...")
        wlan.connect(SSID, PASSWORD)
        while not wlan.isconnected():
            time.sleep(0.5)
    print("Conectado a:", wlan.ifconfig()[0])
```

Esta función activa la interfaz Wi-Fi del ESP32 y espera hasta que se establezca la conexión con el punto de acceso indicado. El método `ifconfig()` devuelve la dirección IP asignada, confirmando la conexión exitosa.

## 3.6. Configuración del cliente MQTT

```
def mqtt_connect_and_subscribe():
    client_id = "esp32-" + str(int(time.time()))
    client = MQTTClient(client_id, AdafRUIT_IO_SERVER,
                        AdafRUIT_IO_PORT, AdafRUIT_USERNAME,
                        AdafRUIT_IO_KEY)
    client.set_callback(mqtt_callback)
    client.connect()
    client.subscribe(topic_for(FEED_LED))
    return client
```

Aquí se crea el cliente MQTT con los datos de usuario, servidor y puerto. La función `set_callback()` asocia la rutina que procesará los mensajes recibidos. El ESP32 se suscribe al feed del LED para poder reaccionar a los comandos enviados desde Adafruit IO.

## 3.7. Recepción de mensajes (Callback)

```
def mqtt_callback(topic, msg):
    t = topic.decode()
    m = msg.decode()
    if FEED_LED in t:
        if m.strip() == '0':
            led.value(0)
        else:
            led.value(1)
```

El *callback* interpreta los mensajes que llegan desde la nube. Cuando el usuario cambia el estado del botón en el dashboard, el ESP32 recibe un valor "0" o "1", y enciende o apaga el LED según corresponda.

## 3.8. Lectura del potenciómetro y publicación de datos

```
v = read_adc_u16(adc)
client.publish(pot_topic, str(v))
```

Cada cierto intervalo, el ESP32 lee el valor analógico del potenciómetro, lo convierte a una escala de 16 bits y lo publica en el feed correspondiente. Este valor puede visualizarse en tiempo real desde el panel de Adafruit IO mediante un bloque tipo *gauge* o *slider*.

## 3.9. Bucle principal y verificación

```
while True:
    client.check_msg()
    now = time.time()
    if now - last_publish >= publish_interval:
        v = read_adc_u16(adc)
        client.publish(pot_topic, str(v))
        last_publish = now
    time.sleep(0.1)
```

El bucle principal mantiene activa la comunicación MQTT, verificando constantemente si hay nuevos mensajes y publicando datos del sensor de forma periódica. Este enfoque garantiza un flujo de información bidireccional entre el dispositivo físico y el entorno web.

Con esta estructura el estudiante puede modificar fácilmente los nombres de los feeds, los intervalos de publicación o incluso incorporar nuevos sensores, ampliando las posibilidades del proyecto IoT con Adafruit IO y ESP32.

# 4. Resultados y Pruebas de Funcionamiento

Una vez completada la configuración de la plataforma Adafruit IO y la carga del programa en el microcontrolador ESP32, se procede a verificar la comunicación bidireccional entre el hardware y la nube. El objetivo de esta fase es comprobar que los valores del potenciómetro se envíen correctamente al servidor y que el control remoto del LED responda a los comandos generados desde el *dashboard*.

## 4.1. Publicación de datos en Adafruit IO

Durante la ejecución, el ESP32 realiza lecturas periódicas del potenciómetro y las publica en el feed configurado. En el panel web de Adafruit IO, este flujo de datos puede visualizarse en tiempo real mediante un bloque tipo *gauge* o gráfico lineal, donde las variaciones del potenciómetro se reflejan en la interfaz.

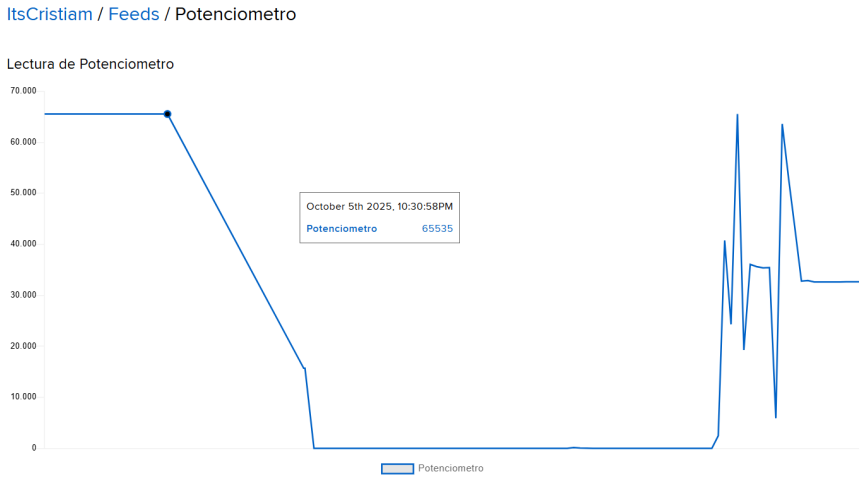



Figura 11: Lectura del potenciómetro publicada en tiempo real en el feed `esp32.potenciometro`.

Cada envío incluye un valor numérico entre 0 y 65535 (equivalente al rango de lectura ADC de 16 bits), el cual se actualiza con una frecuencia determinada por la variable `publish_interval` del código.

## 4.2. Control remoto del LED desde el Dashboard

El bloque de control del LED, configurado como un botón de tipo *toggle*, permite enviar comandos "0" y "1" al feed `esp32.pulsante`. Cuando el usuario cambia el estado del botón en el dashboard, el mensaje MQTT se transmite a la placa, que ejecuta el *callback* para encender o apagar el LED localmente.

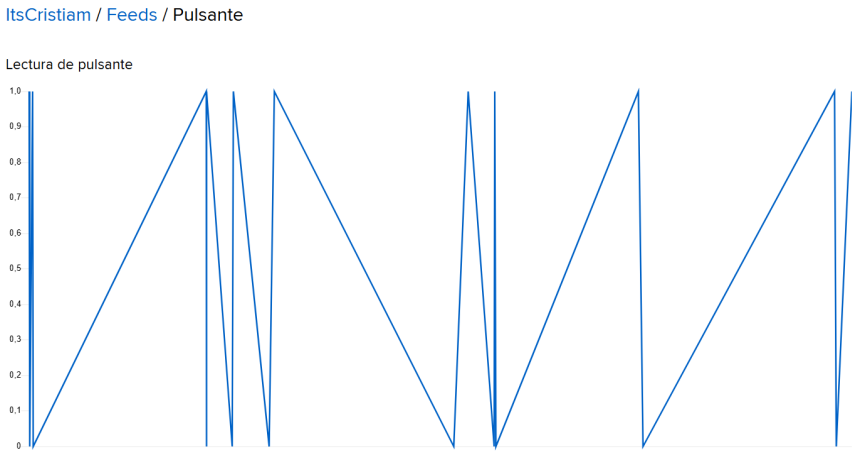



Figura 12: Historial del feed de Adafruit IO con el control del LED remoto.

Este proceso confirma el funcionamiento correcto del canal de suscripción y demuestra la respuesta inmediata del sistema IoT.

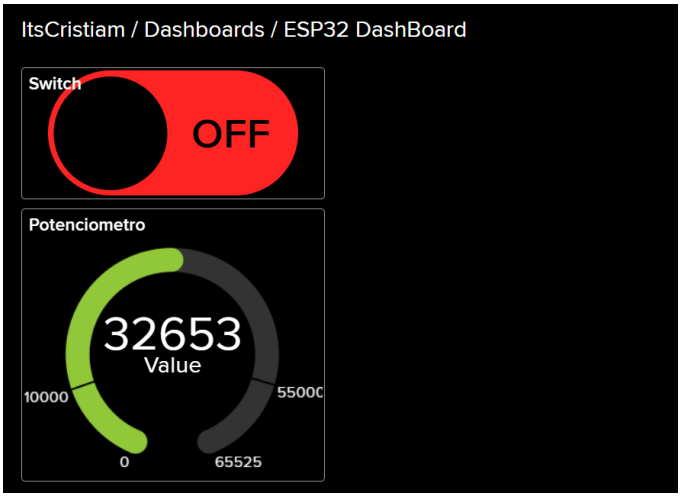



Figura 13: Dashboard de Adafruit IO con los valores actualizados.